Supervised Deep Learning

NECKER

March 2, 2026

Contents

1 Perceptron

1.1 Structure and Purpose

The goal is to separate elements of different classes within the input space using a linear boundary (a straight line). Since the Perceptron computes a weighted sum, it can only divide the space into two distinct half-planes: the points where the linear combination is positive belong to one class, and the others belong to the opposite class. Consequently, the model assumes that the data is **linearly separable**; if the two classes were mixed together, a simple straight line would not be able to divide them without committing errors.

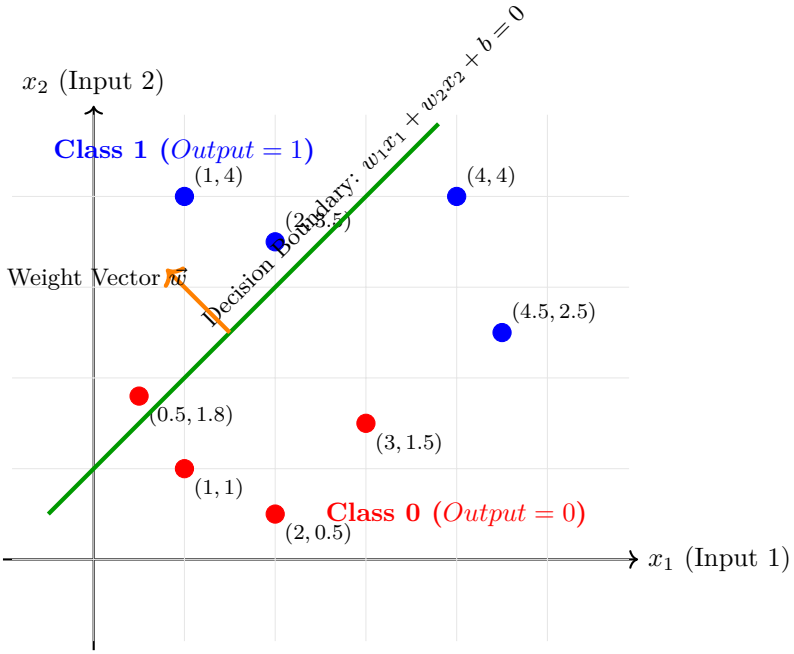
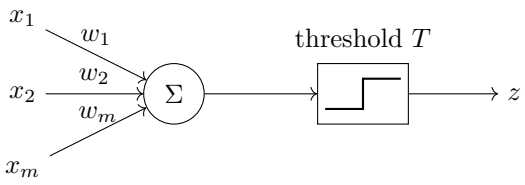


Figure 1: Geometric representation of a Perceptron's decision boundary. The green line divides the space into two half-planes. The weight vector $\vec{w}$ determines the orientation of the line and defines which half-plane corresponds to the positive class.



Activation function:

$$\phi(z(x)) = \begin{cases} 1 & \text{if } z(x) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$z(x) = \sum x_i \cdot w_i - T \ (+b)$$

Where T is the (fixed) threshold and $+b$ is the bias (variable, in a standard perceptron).

Update:

$$w_i = w_i - \mu\big(F(x) - \phi(z(x))\big) \cdot x_i$$
$$b = b - \mu\big(F(x) - \phi(z(x))\big)$$

- μ = learning rate
- $F(x)$ = expected value (label)

- $\phi(z(x))$ = prediction (0 neg, 1 pos)

Note: The correction is heavily influenced by the magnitude of the input.

if x_i is large, then it is very likely the primary cause of the error; thus, multiplying it by the error scaling allows the adjustment to be weighted accordingly (if x_i is close to 0, it is highly likely that it did not cause the error)

1.2 Vectors and Graphical Representation

For simplicity, we will analyze a 2-dimensional scenario to understand how the **Perceptron** operates, but in reality, it employs the exact same mechanism across higher dimensions:

- $W = \{w_1, w_2\}$: weight vector
- $X = \{x_1, x_2\}$: input feature vector
- $W^T = \begin{Bmatrix} w_1 \\ w_2 \end{Bmatrix}$: transposed weight vector (graphically identical, but computationally manipulable)

$$W^T X = \sum_{i=0}^{n} w_i x_i$$

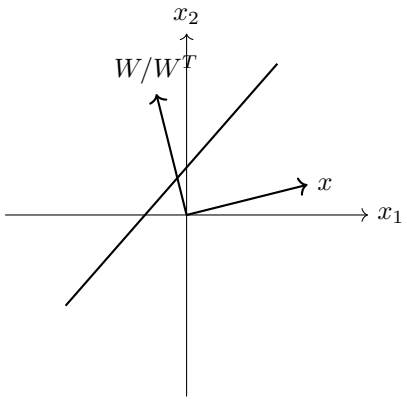
$$w_0 x_0 + w_1 x_1 + b = W^T X \implies \begin{cases} \text{if } \geq 0 = 1 \\ \text{if } < 0 = 0 \end{cases}$$

Let us derive the slope coefficient:

$$w_1 x_1 = -w_0 x_0 - b$$

$$x_1 = -\left(\frac{w_0}{w_1}\right) x_0 - \left(\frac{b}{w_1}\right)$$

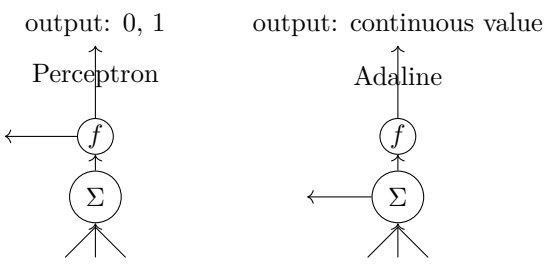
By deriving $m = -\frac{w_0}{w_1}$: (where the first term in parentheses represents m and the second term represents q), we can sketch the graph of our decision boundary line:



If $\mu = 1$, then the correction step shifts the line to accommodate and fit that data point.

1.3 Adaline and Madaline

Adaline: Evolution of the perceptron (previously limited strictly to zero or one outputs, now featuring a variable threshold).



The fundamental distinction is that Adaline does not process all raw outputs identically; instead of computing the error after passing the continuous value through the step function (comparing the label with 0 or 1), it evaluates the error using the raw continuous output itself. Consequently, it executes a larger correction if the value is far from the threshold, and a smaller adjustment if it is close (while the final activation threshold remains unchanged).

Weight correction formula (identical to the previous one):

$$\Delta w_i = \eta \cdot (\text{Error}) \cdot x_i$$

Madaline (Multi-Adaline): Within a multi-layered structure, the system updates only the specific Adaline unit whose weights minimize the current error (a greedy strategy). The final layer (typically a fixed logic gate like an AND gate or a majority voting rule) tells you: "To fix the error and predict the correct output, it would be sufficient for at least one of the hidden-layer Adaline neurons to flip its output from -1 to $+1$."

1.4 Activation Functions and Loss

Activation functions introduce non-linearity into the model, empowering the neural network to learn complex mathematical relationships. Given the total weighted input of the neuron z, defined as:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{1}$$

where w_i represents the weights, x_i denotes the input features, and b is the bias, the resulting output (or activation) of the neuron is expressed as $a = F(z)$.

The primary types of activation functions include:

- **Step Function (Heaviside):** Utilized in the original Perceptron design; it is non-differentiable at $z = 0$ and yields a zero derivative everywhere else.
- **Sigmoid (Logistic):** Squeezes the continuous output into the interval $(0, 1)$, making it interpretable as a probability distribution:
$$\mathrm{F}(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$
- **ReLU (Rectified Linear Unit):** Defined strictly as the maximum value between zero and the raw input:
$$\mathrm{F}(z) = \max(0, z) \tag{3}$$
- **Softplus:** A smooth and completely differentiable variant of the ReLU function (frequently confused with it):
$$\mathrm{F}(z) = \log(1 + e^{z}) \tag{4}$$

1.4.1 Error Computation and Loss Functions

To measure the exact discrepancy between the predicted output of the neuron $a = F(x)$ and the target ground-truth value $\phi(z(x))$, a Loss Function is implemented. When applying the Mean Squared Error (MSE) for an individual data pattern, the loss is defined as:

$$\mathrm{ERR} = \frac{1}{2}\big(F(x) - \phi(z(x))\big)^2 = \frac{1}{2}\big(f(z(x)) - \phi(z(x))\big)^2 \tag{5}$$

Or expressed in its non-compact form:

$$\mathrm{ERR} = \frac{1}{2}\left(y - \frac{1}{1 + e^{-(\sum_{i=1}^{n} w_i x_i + b)}}\right)^2 \tag{6}$$

The goal of the entire training loop is to minimize E by updating weights in the direction opposite to the gradient.

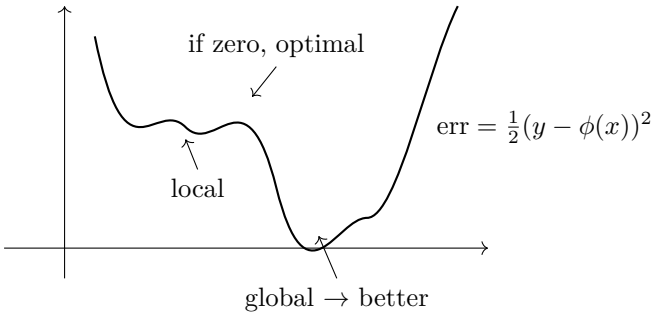
ATTENZIONE: MSE is generally preferred because it penalizes larger errors more aggressively while assigning less weight to smaller deviations.

Once the error is computed, partial derivatives are taken with respect to each weight to calculate the exact rate of error reduction.

This workflow is explained in detail within the Backpropagation chapter.

1.5 Local Minima vs Global Minima

In optimization landscapes, when you find yourself on the slope of a dune regarding the gradient (the derivative of the loss), it implies that adjusting the weight value w_1 causes the error to drop significantly. The linear combination inside the summation $\sum w_i x_i + b$ shifts the input of the sigmoid, steering the network's final output closer to the target y. The optimization algorithm (Backpropagation) tracks the slope of this dune to determine whether it should increase or decrease the value of w_1.



it cannot fall below zero (it makes no sense)

If the gradient evaluates to zero (perfectly flat), we might be trapped in a local minimum rather than the global minimum.

1.6 Entropy and KL Divergence

Entropy and KL Divergence are introduced specifically to circumvent the intrinsic structural limits of **MSE**. To verify the systematic inefficiency of Mean Squared Error in classification tasks paired with sigmoidal activation functions, let us analyze the gradients under MSE.

Suppose the true target is $y = 1$ and the model commits a massive error, producing a heavily skewed output $a = 0.0001$ (the neuron is nearly dormant, when it should be fully active). Let us assume an input value $x_i = 1$ and a standard learning rate $\eta = 0.1$.

The partial derivative of the MSE loss function (the gradient) with respect to a weight w_i contains the derivative of the sigmoid function $a(1 - a)$ due to the chain rule application:

$$\text{Gradient}_{\text{MSE}} = \frac{\partial \text{ERR}}{\partial w_i} = -(y - a) \cdot a(1 - a) \cdot x_i \tag{7}$$

Substituting the numeric values into the separate components:

- **Residual Error:** $(y - a) = (1 - 0.0001) = 0.9999$ (The pure error is nearly at its absolute maximum)
- **Sigmoid Derivative (Saturation):** $a(1 - a) = 0.0001 \cdot (1 - 0.0001) = \mathbf{0.00009999}$

Multiplying these terms together reveals the overall gradient value:

$$\text{Gradient}_{\text{MSE}} = -0.9999 \cdot 0.00009999 \cdot 1 \approx \mathbf{-0.00009998} \tag{8}$$

Let us now calculate the actual update step applied directly to the weight (Δw_i) via the gradient step rule:

$$\Delta w_i = -\eta \cdot \text{Gradient}_{\text{MSE}} = -0.1 \cdot (-0.00009998) = \mathbf{+0.000009998} \tag{9}$$

Geometric Consideration: Despite the model's error being immense (extremely close to 1), the effective weight update step is infinitesimally small (on the order of 10^{-5}). This occurs because the $a(1 - a)$ multiplier acts as a mathematical bottleneck that chokes the slope down to zero. Consequently, the network gets stuck on a flat **plateau** caused by sigmoid saturation, rendering learning completely stationary regardless of how massive the error is.

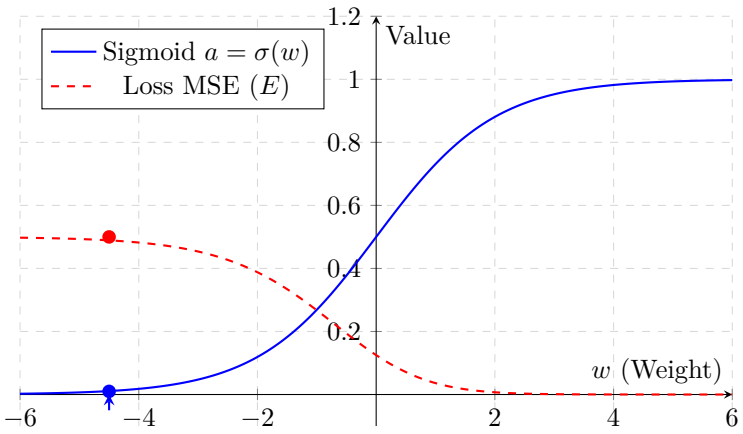


Figure 2: Visualization of gradient saturation utilizing the MSE cost function for a target $y = 1$. When weights push the neuron into the wrong region ($w \to -\infty$), the Loss flattens at its maximum, entirely losing its slope.

KL (Kullback-Leibler) Divergence:

$$\text{KL} = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right) \tag{10}$$

- If $P(i) < Q(i) \to$ the resulting log value becomes negative
- $P(i) =$ true probability distribution (training data target labels)
- $Q(i) =$ estimated probability distribution (network predictions treated as probabilities)

A summation of the distinct informational gaps, used directly as a loss function (always yielding a positive total). By Jensen's Inequality, even if individual log terms evaluate to negative values, the global summation is guaranteed to remain positive.

Cross-Entropy:

$$H(P, Q) = -\sum_i P(i) \log Q(i) \tag{11}$$

This metric also evaluates how far an estimated probability drifts from reality. (The preceding negative sign forces the final output to always be positive).

Core Connection:

$$\text{KL} = H(P, Q) - H(P) \tag{12}$$

Where $H(P)$ represents the raw entropy of the true distribution P (which acts as a constant value).

Binary Cross-Entropy (BCE): (The standard loss choice for binary classification tasks).

- $y_i \in (0, 1) =$ predicted probability emitted by the model
- $d_i \in (0, 1) =$ ground-truth target label

$$\text{BCE} = -d_i \log(y_i) - (1 - d_i) \log(1 - y_i) \tag{13}$$

It penalizes incorrect classifications significantly more aggressively than Mean Squared Error.

Applied Numerical Example:
Consider a binary classification setup where the true target belongs to Class 1. The two distinct probability distributions are mapped as:

- **True Reality (P):** $P(1) = 1$ and $P(0) = 0$ (Absolute certain distribution, fixed target)
- **Model Prediction (Q):** $Q(1) = y$ and $Q(0) = 1 - y$ (Current neuron output value)

Let us evaluate the Entropy of reality $H(P)$ to prove it acts as a zero constant:

$$H(P) = -\sum_i P(i) \log P(i) = -\big(1 \cdot \log(1) + 0 \cdot \log(0)\big) = 0 \tag{14}$$

Since $H(P) = 0$, substituting this into the equation $KL = H(P,Q) - H(P)$ dictates that under this ideal scenario $KL = H(P,Q)$.

Let us numerically evaluate the behavioral dynamics of the Loss across two polar opposite scenarios:

1. **Scenario A (The model predicts correctly):** The network is 90% confident in the correct classification $\rightarrow y = 0.9$.
   - **Cross-Entropy:** $H(P,Q) = -\big(1 \cdot \log(0.9) + 0 \cdot \log(0.1)\big) = -(-0.105) = \mathbf{0.105}$
   - **KL Divergence:** $\text{KL} = \sum_i P(i) \log\left(\frac{P(i)}{Q(i)}\right) = 1 \cdot \log\left(\frac{1}{0.9}\right) + 0 = \mathbf{0.105}$

   Result: Both metrics yield a low score (near zero), demonstrating that the predicted distribution closely matches reality.

2. **Scenario B (The model fails drastically):** The network assigns a meager 10% confidence to the correct class $\rightarrow y = 0.1$.
   - **Cross-Entropy:** $H(P,Q) = -\big(1 \cdot \log(0.1) + 0 \cdot \log(0.9)\big) = -(-2.302) = \mathbf{2.302}$
   - **KL Divergence:** $\text{KL} = 1 \cdot \log\left(\frac{1}{0.1}\right) + 0 = \mathbf{2.302}$

   Result: The calculated loss and informational penalty skyrocket exponentially (from 0.105 to 2.302), creating a massive gradient that forces the weights to correct themselves rapidly during Backpropagation.

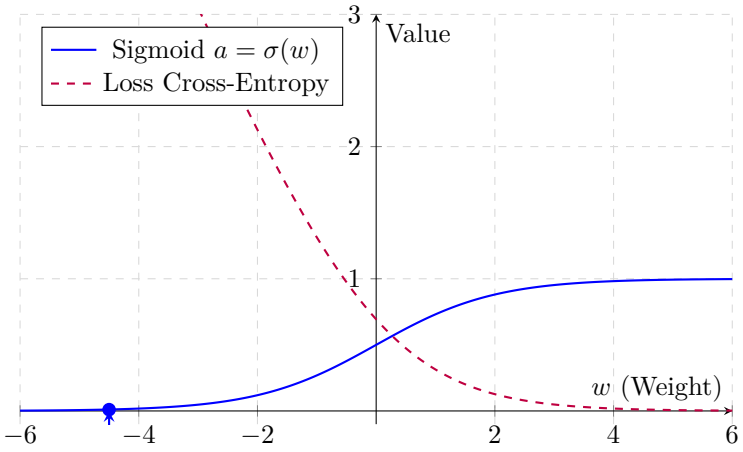


Figure 3: Visualization of Cross-Entropy loss behavior for a target $y = 1$. When weights drive the neuron into the incorrect region ($w \rightarrow -\infty$), the curve never plateaus; instead, it scales up toward infinity, securing a continuous, strong gradient.

2 back propagation

Backpropagation is the fundamental optimization strategy applied to execute weight corrections for every distinct neuron across a architecture. Up to this point, we have evaluated the mechanics of a singular decoupled node; in production architectures, we orchestrate large networks of neurons simultaneously, stacked in parallel and distributed across consecutive hidden layers. Because the entire network yields a solitary unified prediction at the terminal layer, the **Loss** can only be computed explicitly at that endpoint, requiring the error signals to be propagated backward through the prior layers.

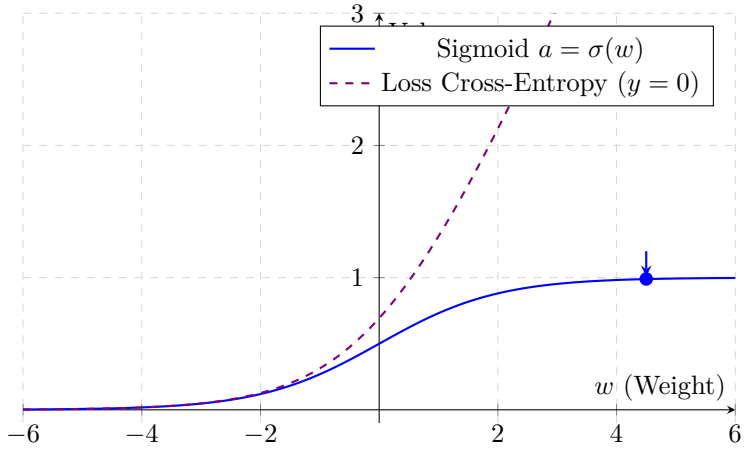
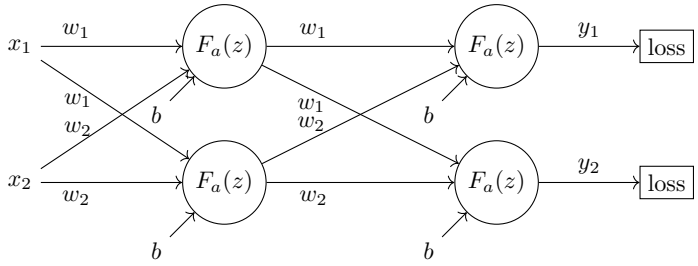


Figure 4: Cross-Entropy representation for a target $y = 0$. If weights grow positively ($w \to +\infty$), the neuron's prediction reaches 1 (maximum error). The loss function counters this with a symmetrical steep slope, forcing the optimizer to quickly change course.



PROCEDURE:

Terminal Layer:

- Pre-activation calculation for all network neurons: $z = w_1x_1 + w_2x_2 + b$
- Activation output: $F_a(z) = \text{ReLU}(z)$, which represents our final prediction y
- $\text{loss}(F_a(z)) = \frac{1}{2}(\hat{y} - y)^2$ (this calculation is restricted explicitly to the final layer)

Therefore:

$$\text{Loss} = \frac{1}{2}\left(\hat{y} - \overbrace{\text{ReLU}(\underbrace{w_1x_1 + w_2x_2 + b}_{z})}^{y}\right)$$

Let us derive the partials:

- Loss gradient with respect to $F_a(z) \to \frac{\partial \text{loss}}{\partial y}$
- Activation gradient with respect to $z \to \frac{\partial y}{\partial z}$
- Pre-activation gradient with respect to $w_1, w_2, b \to \frac{\partial z}{\partial w_1}, \frac{\partial z}{\partial w_2}, \frac{\partial z}{\partial b}$

Chain Rule: To isolate the exact influence of a distinct weight or bias on the final cost, we must chain the local partial derivatives along the composite function pathway:

$$\frac{\partial \text{loss}}{\partial w} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w} \tag{15}$$

By utilizing the chain rule, we compute the explicit adjustments needed to execute weight updates.

Weights and Bias Gradients:

$$\Delta w_1 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$
$$\Delta w_2 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_2}$$
$$\Delta b = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Weights and Bias Parameter Updates:

$$w_1 = w_1 - \mu \cdot \Delta w_1$$
$$w_2 = w_2 - \mu \cdot \Delta w_2$$
$$b = b - \mu \cdot \Delta b$$

(μ = learning rate)

Preceding Layer Neurons:

At this stage of backpropagation, the only missing component is the explicit derivative of the loss function, since intermediate layers lack direct target coordinates ($\hat{y}$). Now that we can express the incoming gradient as:

$$\frac{\partial \text{loss}}{\partial y} = \sum_{i=1}^{m} w_i \frac{\partial \text{loss}}{\partial y_i} \tag{16}$$

For instance, evaluated across two paths:

$$\left(w_1 \frac{\partial \text{loss}}{\partial y_1} + w_2 \frac{\partial \text{loss}}{\partial y_2} \right) \tag{17}$$

WARNING: The synaptic weights used in this summation step must be the un-updated values from the current forward pass.

- From this point, we proceed with calculations exactly as before: $\Delta w_1 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_1}$, $\Delta w_2 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_2}$, $\Delta b = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$, and then perform parameter updates.

Analytical comparison with historical updating structures:

$$w = w - \mu(F(y) - \phi(z)) \cdot x \quad \text{vs} \quad w = w - \mu \left(\frac{\partial \text{loss}}{\partial w} \right) \tag{18}$$

$$b = b - \mu(F(y) - \phi(z)) \quad \text{vs} \quad b = b - \mu \left(\frac{\partial \text{loss}}{\partial b} \right) \tag{19}$$

$$\frac{\partial \text{loss}}{\partial w} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w} \quad \text{(the feature term } x \text{ is naturally extracted during the } \partial z/\partial w \text{ derivative step)} \tag{20}$$

$$\frac{\partial \text{loss}}{\partial b} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b} \quad \text{(the derivative } \partial z/\partial b \text{ consistently evaluates to } = 1\text{, making it independent of } x\text{)} \tag{21}$$

Analytical Derivation of Gradients with Respect to z: Given the basic linear combination of a neuron node: $z = w_1 x_1 + w_2 x_2 + b$

When executing partial differentiation, all variables isolated from the explicit variable of interest are mathematically treated as fixed constants (meaning their respective derivatives resolve to zero):

- **Derivative with respect to weight w_1:** The parameter is linked multiplicatively to input feature x_1. The remaining block $(w_2 x_2 + b)$ decays to 0:

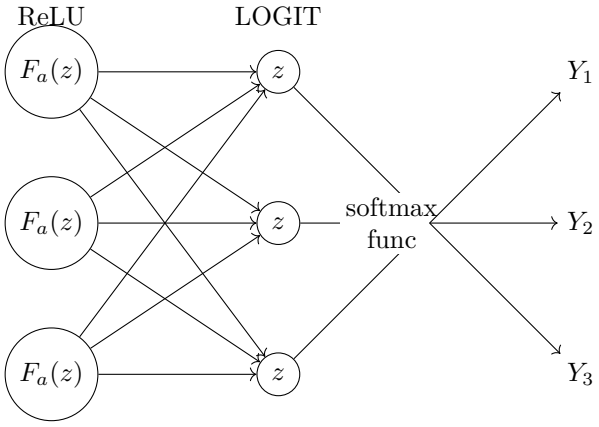
$$\frac{\partial z}{\partial w_1} = \frac{\partial}{\partial w_1}(w_1 x_1) + 0 = x_1 \tag{22}$$

- **Derivative with respect to bias b:** The bias acts as an isolated additive term decoupled from input feature matrices. The component $(w_1 x_1 + w_2 x_2)$ drops to 0, while the derivative of a variable with respect to itself resolves to unity:

$$\frac{\partial z}{\partial b} = 0 + \frac{\partial}{\partial b}(b) = \mathbf{1} \tag{23}$$

Geometric Consequence: In their respective optimization update steps (Δw and Δb), input x acts as a scaling multiplier exclusively for weight updates (quantifying its baseline responsibility for the error), whereas it drops out completely for bias terms, whose updates depend solely on the error accumulated downstream of the neuron.

2.1 with soft max



Softmax Activation:

$$\hat{Y}_i = \frac{\exp(z_i)}{\sum_{j=1}^{c} \exp(z_j)} \tag{24}$$

- $\exp(z_i) \rightarrow$ Exponentiation emphasizes and exaggerates the relative differences among raw z values.
- $\sum_{j=1}^{c} \exp(z_j) \rightarrow$ The denominator ingests the entire array of z values for normalization.
- $\hat{Y}_i \rightarrow$ Translates to a probability calculation matching the structure of standard classical probability formatting: $\frac{z}{\text{total } z}$.

- $c =$ number of classes
- $Y_i =$ ground-truth label
- $\hat{Y}_i =$ network prediction probability

Loss Computation:

When preparing for **Backpropagation**, incorporating a Softmax activation fundamentally reshapes the gradient derivation workflow.

- **ReLU / Sigmoid Pathways:** Raw values z pass independently into activation layers to yield prediction y (making y dependent entirely on its corresponding localized z).
- **Softmax Activation:** To generate prediction y, the system processes the global pool of logits z emitted across all output neurons through a shared Softmax layer, returning a normalized probability distribution across all classes (yielding as many y outputs as there are input z values, since each logit must occupy the numerator once).

Given these global dependencies, we must clarify two core optimization conventions:

- In Softmax configurations, Cross-Entropy serves as the standard Loss function: $\left(-\sum_{i=1}^{c} Y_i \log(\hat{Y}_i)\right)$.
- The labels are encoded as a sparse one-hot vector containing zeros everywhere except for a single 1 indicating the correct target class (e.g., $[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]$).

Therefore, the loss function simplifies to: $-\left(0 \cdot \log(\hat{Y}_1) + 0 \cdot \log(\hat{Y}_2) + 1 \cdot \log(\hat{Y}_s) \dots\right)$

Which reduces directly to:

$$\text{loss} = -\log(\hat{Y}_i) \implies \frac{\partial \text{loss}}{\partial \hat{Y}_i} = -\frac{y_i}{\hat{Y}_i} = -\frac{1}{\hat{Y}_i} \tag{25}$$

(Recall the mathematical identity: $\frac{d}{dx} \log x = \frac{1}{x}$)

Calculating Logit Weights in Relation to Global Loss:

Because we must initialize backpropagation from the target prediction y_i and step backward, we trace the derivative of the Softmax function $\left(\frac{\partial \hat{Y}_i}{\partial z_i}\right)$ across every logit z_i:

Theoretically, one would evaluate all combination paths for every predicted probability $\hat{Y}$ across all existing logits z. For example, given an environment with three logits z: $\frac{\partial \hat{Y}_1}{\partial z_1}, \frac{\partial \hat{Y}_1}{\partial z_2}, \frac{\partial \hat{Y}_1}{\partial z_3}$ — $\frac{\partial \hat{Y}_2}{\partial z_1}, \frac{\partial \hat{Y}_2}{\partial z_2}, \frac{\partial \hat{Y}_2}{\partial z_3}$ — $\frac{\partial \hat{Y}_3}{\partial z_1}, \frac{\partial \hat{Y}_3}{\partial z_2}, \frac{\partial \hat{Y}_3}{\partial z_3}$. These components would then be aggregated according to partial derivative summation laws, yielding 3 final gradients (1 for each prediction $\hat{Y}$).

HOWEVER, we know that the general derivative of Softmax with respect to logit z_i resolves into two distinct cases:

Case 1: Where $\hat{Y}_i$ and $z_j \to i = j$ (z corresponds directly to the neuron of interest)

$$\frac{e^{z_i} \cdot \left(\sum_{k=1}^{c} e^{z_k}\right) - e^{z_i} \cdot e^{z_i}}{\left(\sum_{k=1}^{c} e^{z_k}\right)^2} = \frac{e^{z_i}\left[\left(\sum_{k=1}^{c} e^{z_k}\right) - e^{z_i}\right]}{\left(\sum_{k=1}^{c} e^{z_k}\right)^2} = \frac{e^{z_i}}{\sum_{k=1}^{c} e^{z_k}} \cdot \frac{\sum_{k=1}^{c} e^{z_k} - e^{z_i}}{\sum_{k=1}^{c} e^{z_k}} \tag{26}$$

$$= \hat{Y}_i \cdot \left(\frac{\sum e^{z_k}}{\sum e^{z_k}} - \frac{e^{z_i}}{\sum e^{z_k}}\right) = \hat{Y}_i \cdot (1 - \hat{Y}_i) \tag{27}$$

Note: To compute the Softmax derivative, we apply the calculus quotient rule: $dx = \frac{(\text{num derivative}) \cdot \text{den} - \text{num} \cdot (\text{den derivative})}{(\text{den})^2}$

Case 2: Where $\hat{Y}_i$ and $z_j \to i \neq j$ (z does NOT correspond to the neuron of interest)

$$\frac{0 \cdot \sum_{k=1}^{c} e^{z_k} - e^{z_i} \cdot e^{z_j}}{\left(\sum_{k=1}^{c} e^{z_k}\right)^2} = -\frac{e^{z_i}}{\sum_{k=1}^{c} e^{z_k}} \cdot \frac{e^{z_j}}{\sum_{k=1}^{c} e^{z_k}} = -\hat{Y}_i \cdot \hat{Y}_j \tag{28}$$

Why does the numerator evaluate to e^{z_i} when $i = j$, but drops to 0 when $i \neq j$? This occurs because the mathematical term survives differentiation exclusively if the logit indices align. Proof:

- $i = j : \hat{Y}_i = \frac{e^{z_i}}{e^{z_i} + \sum_{k \neq i} e^{z_k}} \to \frac{\partial \hat{Y}_i}{\partial z_i}$ = derivative of $\frac{e^{z_i}}{e^{z_i} + K}$ (K acts as a fixed constant because during composite differentiation, non-target variables are treated as constant values)
- $i \neq j : \hat{Y}_i = \frac{e^{z_i}}{e^{z_j} + \sum_{k \neq j} e^{z_k}} \to \frac{\partial \hat{Y}_i}{\partial z_j}$ = derivative of $\frac{K}{e^{z_j} + K}$ (where the derivative of the numerator resolves to zero)

Consequently, the derivative of Softmax separates neatly into two operational paths:

- If $i = j \to \hat{Y}_i(1 - \hat{Y}_i)$
- If $i \neq j \to -\hat{Y}_i \hat{Y}_s$

Applying the chain rule: $\frac{\partial \text{loss}}{\partial z_i} = \frac{\partial \text{loss}}{\partial \hat{Y}_i} \cdot \frac{\partial \hat{Y}_i}{\partial z_i}$
Chaining this with the loss derivative $\frac{\partial \text{loss}}{\partial \hat{Y}_s} = -\frac{1}{\hat{Y}_s}$ yields two distinct scenarios:

- If $i = j \to \hat{Y}_i \cdot (1 - \hat{Y}_i) \cdot \left(-\frac{1}{\hat{Y}_i}\right) = -(1 - \hat{Y}_i) = \hat{Y}_i - 1$
- If $i \neq j \to -\hat{Y}_i \hat{Y}_j \cdot \left(-\frac{1}{\hat{Y}_i}\right) = \hat{Y}_j$

Since index i corresponds explicitly to the correct target label, we observe that:

- For the true target path ($i = j$), the descent gradient is expressed as: $\hat{Y}_i - 1$ (meaning the value is either zero or strictly negative, ensuring continuous downward descent).
- For incorrect paths, the gradient evaluates to: $\hat{Y}_i$ (which remains positive, constantly driving down its own overconfident activation by the exact amount of its current miscalculation).

Finally, we evaluate $\frac{\partial z_i}{\partial w_j}$ and solve for weight update values Δw using:

$$\boxed{\frac{\partial \text{loss}}{\partial w_i} = \frac{\partial \text{loss}}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_i}}$$

3 momentum method

The core purpose of incorporating momentum is to accelerate optimization updates if gradient steps are consistently heading in an identical direction, while damping updates if corrections experience heavy oscillation (mimicking the physical properties of a ball rolling down a valley).

Classical Optimization Update Format:

$$w = w - \mu \Delta w \tag{29}$$

Momentum Upgraded Format:

$$v_t = \beta v_{t-1} - \mu \Delta w_t$$
$$w_t = w_{t-1} + v_t$$

(where velocity vector v_t is computed and integrated below)
β = momentum damping coefficient

The term v_{t-1} tracks the velocity vector accumulated during the previous parameter update, with baseline condition $v_0 = 0$ (causing the inaugural update step to simplify cleanly to $w = w - \mu \Delta w$).

Operational Pipeline:

- First, update the velocity vector based on the current step position coordinates.
- Next, update the parameter weights utilizing the newly computed velocity vector.

4 nesterov momentum

Nesterov Accelerated Gradient achieves higher precision because it factors in the cumulative nature of velocity. Specifically:

1. It projects the future position of the weight parameter by applying the current velocity vector beforehand.
2. It evaluates the loss gradient explicitly at this predicted look-ahead weight position (allowing the system to proactively brake and slow down if it is moving fast toward an ascending slope, before the standard loss actually rises).
3. It updates the true weight parameter leveraging this optimized velocity calculation.

Structural update formulas for execution:

$$\tilde{w} = w_t + \beta v_{t-1} \qquad (\beta = \text{momentum coefficient})$$
$$v_t = \beta v_{t-1} - \mu \cdot \left(\frac{\partial \text{loss}}{\partial \tilde{w}} \right)$$
$$w_{t+1} = w_t + v_t$$

Step-by-Step Breakdown:

- $\tilde{w}$ maps the look-ahead prediction of the new weight value relying on prior velocity v_{t-1} (acting as a parameter update step bypass that skips raw gradient calculation temporarily).
- We evaluate v_t based on this look-ahead prediction (computing the loss gradient directly at position $\tilde{w}$ to measure how far our velocity-driven prediction drifts from the target).
- We execute the actual weight update (w_{t+1}) using this fine-tuned velocity vector.

5 batch

Now that we understand how parameter updates are computed, we must determine when to execute them. Computing weight updates after processing every single data entry across a massive dataset is highly inefficient. To address this, the data is partitioned into discrete segments called **batches**.

Batch: A grouped sub-segment of the full dataset. Example: $D = \left\{ \overbrace{(x_1, y_1), (x_2, y_2)}^{\text{batch 1}}, \overbrace{(x_3, y_3), (x_4, y_4)}^{\text{batch 2}} \right\}$

- **Batch Loss** = The mean value of the loss computed across all distinct training training instances within that specific batch (used to calculate the localized batch step gradient).
- **Total Loss** = The overall mean loss evaluated across all processed batches within a single training epoch, serving as a baseline performance metric to determine if the network is converging.

$$\text{total loss} = \text{average of all distinct batch losses}$$

- Small Batch Size: + Higher variance observed between successive batch updates.
- Large Batch Size: - Reduced variance observed between successive batch updates.

6 discrete convolution

Discrete convolution represents the foundational processing layer an input image undergoes within a convolutional neural network. It functions by filtering the image to accentuate salient structural details (with feature selection criteria adapting dynamically based on how the network is trained).

$$\boxed{y = \sum x(a) \cdot h(n-a) = (x \cdot h) \cdot n}$$

- $\sum x(a)$ demonstrates that the mathematical operator is iterating over the pixel spatial coordinate array of the image ($x(a)$).
- $h(n-a)$ = kernel / filter $\rightarrow$ formatted notation indicating that the sliding matrix shifts sequentially across all image pixel coordinates.

EXAMPLE:
When computing the output pixel value mapped precisely at spatial coordinate index $n = 3$, the core equation is formalized as:

$$y(3) = \sum_a x(a) \cdot h(3-a) \tag{30}$$

Expanding this summation over the local coordinate neighborhood of the target pixel (assuming, for instance, coordinate range $a \in \{2, 3, 4\}$), we can analytically track how the sliding operator isolates a specific region of the input matrix:

$$\begin{aligned}
&\text{For } a = 2 \implies x(2) \cdot h(3-2) = x(2) \cdot h(1) \quad (\textit{preceding pixel} \times \textit{rightmost kernel weight}) \\
&\text{For } a = 3 \implies x(3) \cdot h(3-3) = x(3) \cdot h(0) \quad (\textit{central pixel} \times \textit{center kernel weight}) \\
&\text{For } a = 4 \implies x(4) \cdot h(3-4) = x(4) \cdot h(-1) \quad (\textit{succeeding pixel} \times \textit{leftmost kernel weight})
\end{aligned}$$

The algebraic summation of these three discrete localized computations yields the final value mapped into coordinate $y(3)$.

CORE LOGIC:

- The filter/kernel is a specialized matrix designed to detect specific spatial patterns (such as intensity homogeneity or contrasts between adjacent pixels).
- This matrix is applied systematically across all local pixel neighborhoods, aggregating the resulting products into a single sum.

Example:

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \tag{31}$$

→ Sobel Filter (specialized for edge detection). Each matrix coordinate maps to a neighboring pixel location, where the central zero aligns with the target pixel under evaluation.

- The newly computed pixel value is stored within a fresh, isolated copy of the image canvas.
- Result: Localized pixel gradients are amplified, making structural edges and boundaries stand out sharply.

3×3 **Kernels vs** 9×9 **Kernels:**

- 3×3 matrices are computationally much cheaper, requiring only 9 multiplications per pixel compared to the 81 multiplications demanded by a 9×9 layout.
- 3×3 filters offer higher spatial saliency; larger 9×9 kernels evaluate an overly broad area, which tends to average out and blur highly localized details.

6.1 stride, padding and pooling

1. **Stride:** Defines the exact number of pixel coordinates the sliding filter skips as it advances across the image.
2. **Padding:** The process of appending extra rows and columns of boundary pixels (typically zeros) around the perimeter of the input canvas. This prevents the image from shrinking after convolution and ensures edge pixels are fully captured (without padding, a 3×3 filter could not evaluate the outermost pixel layers as centers).

$$\boxed{n_{\text{step}} = \frac{F-1}{2}} \quad \text{where } F = \text{dimension of the square kernel matrix}$$

3. **Pooling:** Executes spatial dimensionality reduction on the feature map. It passes a sliding window across the canvas using one of two primary strategies:
   - **Max Pooling:** $\begin{array}{|c|c|}\hline 1 & 3 \\ \hline 4 & 8 \\ \hline \end{array} = 8$ (isolates and extracts exclusively the maximum value)
   - **Mean Pooling:** $\begin{array}{|c|c|}\hline 1 & 3 \\ \hline 4 & 8 \\ \hline \end{array} = 4$ (computes the mathematical average of all values in the window)

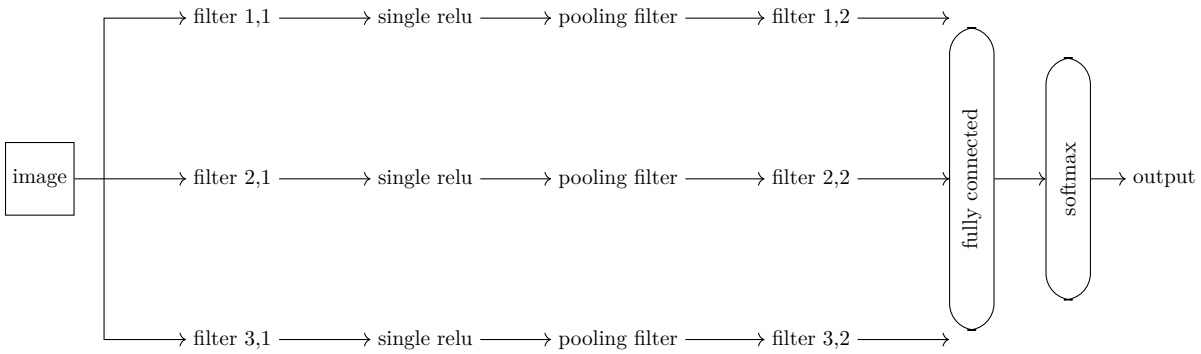
• While 2D filters process data along three structural dimensions (handling grayscale single-channel layers or RGB triple-channel profiles), they compress the result into a purely 2D output slice representing that specific filter's feature map.

$$\boxed{W = [D, D, I, F]}$$

D = kernel matrix dimensions (width / height)
I = number of incoming image channels / layers
F = number of distinct filters applied

This structural arrangement shapes W as a 4D filter tensor.
If an architecture employs 6 distinct filters simultaneously, it outputs 6 independent 2D feature maps, which can then serve as stacked input channels for subsequent convolutional operations (refer to lecture 8, slide 70).

6.2 image processing pipeline



- Parallel Filtering: Employs an array of structurally diverse kernels simultaneously to detect distinct low-level visual traits (e.g., edges, specific color gradients, geometric points).

- Sequential Filtering: Arranges filters in a consecutive pipeline to construct increasingly specialized and complex feature representations (e.g., primary layer → simple lines, secondary layer → complex squares).

In short:

- The convolutional filter isolates and highlights a target spatial trait.
- The subsequent ReLU layer maps and learns to register its presence.

Filter Weights and Bias Integration:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \tag{32}$$

These 9 discrete matrix values function directly as trainable synaptic weights. Once the new pixel value is aggregated via dot-product convolution, a scalar bias is added.

During backpropagation, this entire spatial grid is optimized exactly like a standard neural node, fine-tuning its weights and bias values.

7 depth and vanishing gradient

Network **depth** refers to the total volume of stacked hidden layers within the architecture. As we introduce more layers sequentially, the backward propagation of error encounters systematic degradation. This bottleneck is caused by the vanishing gradient phenomenon, where layers closest to the input experience extremely slow parameter optimization compared to layers near the output.

7.1 vanishing gradient:

This issue severely impacts bounded activation functions like Sigmoid or Tanh (though Tanh is slightly less vulnerable). We can analyze this bottleneck by evaluating the mathematical derivative of the Sigmoid function:

$$\underbrace{\underbrace{\frac{1}{1+e^{-x}}}_{\text{bounded between 0 and 1}} \cdot \left(1 - \frac{1}{1+e^{-x}}\right)}_{\text{always } \leq 0.25} \tag{33}$$

As gradients flow backward through deep layers, the error signal is continuously squashed. Consider updating a weight situated at the very first layer (closest to raw input). To compute its gradient, the chain rule must multiply partial derivatives backward across the entire sequence of hidden layers all the way from the terminal Loss:

$$\frac{\partial \text{Loss}}{\partial w_{\text{layer1}}} = \frac{\partial \text{Loss}}{\partial y_4} \cdot \underbrace{\frac{\partial y_4}{\partial z_4}}_{\text{Sigmoid 4}} \cdot \frac{\partial z_4}{\partial y_3} \cdot \underbrace{\frac{\partial y_3}{\partial z_3}}_{\text{Sigmoid 3}} \cdot \frac{\partial z_3}{\partial y_2} \cdot \underbrace{\frac{\partial y_2}{\partial z_2}}_{\text{Sigmoid 2}} \cdot \frac{\partial z_2}{\partial y_1} \cdot \underbrace{\frac{\partial y_1}{\partial z_1}}_{\text{Sigmoid 1}} \cdot \frac{\partial z_1}{\partial w_{\text{layer1}}} \tag{34}$$

Even if the terminal error signal ($\frac{\partial \text{Loss}}{\partial y_4}$) is massive (e.g., the model produced an entirely incorrect prediction), look at what happens as it passes through the successive bottlenecks of bounded activations. Under standard conditions, each Sigmoid derivative outputs a maximum value of 0.25. Isolating the activation derivatives along the chain reveals the exponential decay:

$$\text{Gradient} \propto \text{Loss} \cdot 0.25 \cdot 0.25 \cdot 0.25 \cdot 0.25 \tag{35}$$

$$\text{Gradient} \propto \text{Loss} \cdot (0.25)^4 \tag{36}$$

8 the dying ReLU problem and exploding gradients

8.1 dying neurons:

This optimization bottleneck specifically impacts **ReLU** neurons due to the function's piece-wise definition, which maps inputs as follows:

$$f(z) = \max(0, z) = \begin{cases} z & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases} \tag{37}$$

Its first derivative with respect to input z (the slope of the activation profile) maps directly to a standard step function:

$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases} \tag{38}$$

Engineering Note: Because the derivative evaluates to a constant 1 for all positive inputs, ReLU does not squash or choke the error signal during Backpropagation. The gradient flows backward completely intact, avoiding the exponential decay intrinsic to Sigmoidal profiles.

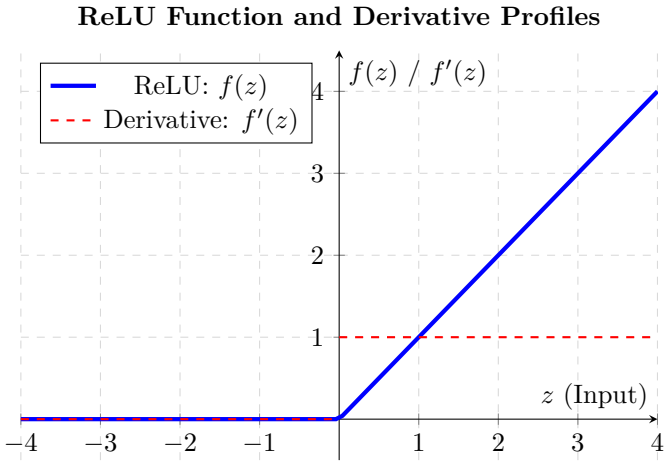


Figure 5: Geometric visualization of the piece-wise ReLU function (solid blue line) and its corresponding partial derivative (dashed red line).

However, if a neuron's input drops below zero ($z < 0 \Rightarrow y = 0$), its gradient becomes permanently zero, rendering the neuron "dead." As network depth scales up, there is an exponentially higher probability that large clusters of neurons will deactivate permanently, rendering them completely useless for feature extraction.

8.2 exploding gradient:

Although ReLU maintains a constant derivative of 1 for positive inputs ($z > 0$), deep architectures remain highly vulnerable to **Exploding Gradients**. This occurs because the *Chain Rule* continuously multiplies the interconnected **synaptic weights** (w) across successive layers:

$$\frac{\partial \text{Loss}}{\partial w_1} = \frac{\partial \text{Loss}}{\partial y_4} \cdot \underbrace{\frac{\partial y_4}{\partial z_4}}_{1} \cdot \underbrace{\frac{\partial z_4}{\partial y_3}}_{\mathbf{w_4}} \cdot \underbrace{\frac{\partial y_3}{\partial z_3}}_{1} \cdot \underbrace{\frac{\partial z_3}{\partial y_2}}_{\mathbf{w_3}} \cdot \underbrace{\frac{\partial y_2}{\partial z_2}}_{1} \cdot \underbrace{\frac{\partial z_2}{\partial y_1}}_{\mathbf{w_2}} \cdot \underbrace{\frac{\partial y_1}{\partial z_1}}_{1} \cdot \frac{\partial z_1}{\partial w_1} \tag{39}$$

Isolating the layer-to-layer transitions shows that the final gradient is directly proportional to the geometric product of the intermediate weights:

$$\text{Gradient} \propto \text{Loss} \cdot 1 \cdot w_4 \cdot 1 \cdot w_3 \cdot 1 \cdot w_2 \cdot 1$$
$$\text{Gradient} \propto \text{Loss} \cdot (w_4 \cdot w_3 \cdot w_2)$$

Conclusion: If intermediate weights are initialized or updated to values even slightly greater than 1 (e.g., $|w| > 1$), their continuous multiplication scales up exponentially with depth. This triggers massive, volatile weight updates that destabilize training. To mitigate this, a technique called **Gradient Clipping** is applied, forcing gradients back within a safe, predefined threshold if they exceed maximum limits.

9 skip connection (res net)

Instead of forcing the network layers to approximate the direct mapping $F(x) = H(x) - x$ (the delta between the target output and the current layer representation),
we re-formulate the target mapping as $H(x) = F(x) + x$

Where:

- $H(x)$ = the ideal underlying target mapping
- $F(x)$ = the residual mapping, representing what the current layer still lacks to reach the optimum
- x = the original input identity routed directly past the layer

This architectural adjustment is exceptionally powerful for mitigating vanishing gradients. To conceptualize this formulation, think of $F(x)$ as a feature map after undergoing multiple convolutions minus the initial input map (the net change Δ). $H(x)$ represents the optimized target image reconstructed from those altered pixel profiles.

Behavior During Backpropagation:

$$\boxed{\frac{\partial \text{loss}}{\partial x} = \frac{\partial \text{loss}}{\partial H(x)} \cdot \left(\frac{\partial F(x)}{\partial x} + 1 \right)}$$

This is derived directly by differentiating the residual block $\overbrace{F(x) + x}^{H(x)} \Rightarrow \partial F(x) + 1$.

This constant +1 term prevents the gradient from decaying toward zero, ensuring the original input information flows backward intact across the entire network architecture.

Consequently:

- If $H(x) - x = 0 \rightarrow F(x) = 0$, the network simply learns an identity mapping, passing the data through unchanged (ideal when the input is already close to the optimal representation).
- If $H(x) - x \neq 0 \rightarrow F(x) \neq 0$, the network actively optimizes the residual weights to refine the feature maps.

By combining this mechanism with gradient clipping (applied at every layer), deep architectures can be trained effectively without suffering from vanishing gradients.

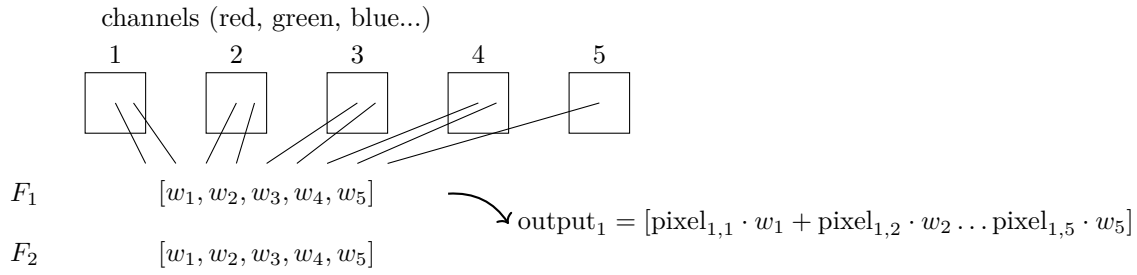
10 Potential Bottlenecks and Architectural Challenges

10.1 channel reduction

We have observed how applying multiple convolutional filters in parallel allows us to increase channel depth. However, if an intermediate layer outputs 64 channels and we need to scale them down to 32, how do we reduce this depth? We implement 1×1 convolutions.

In this configuration: We apply 32 distinct filters, where each filter contains 64 individual weights (one assigned to each incoming channel).

In practice, this performs a weighted blend across the existing layers, compressing channel depth.



This technique enables the model to extract complex cross-channel correlations. For example, a newly generated channel might combine information regarding red color profiles with horizontal edge data. Because every individual pixel in the output map is a weighted average across all 64 original channels, the network automatically learns optimal channel selection and dimensionality reduction during training.

10.2 the rotation problem

If a target object is presented at a rotated angle during inference, a standard network may fail to recognize it. This limitation occurs because traditional convolutional layers are highly sensitive to the absolute spatial coordinates of pixels.

Solution:

Implement Data Augmentation. For every training image, populate the dataset with multiple copies rotated at diverse angles.

10.3 multi-task learning architectures

Neural network architectures can be optimized to execute multiple distinct objectives simultaneously. For instance, a model can be trained to classify an object while tracking its precise coordinates (drawing a bounding box around a specific flower or dog). How can a network handle this if its traditional architecture is geared toward a single objective?

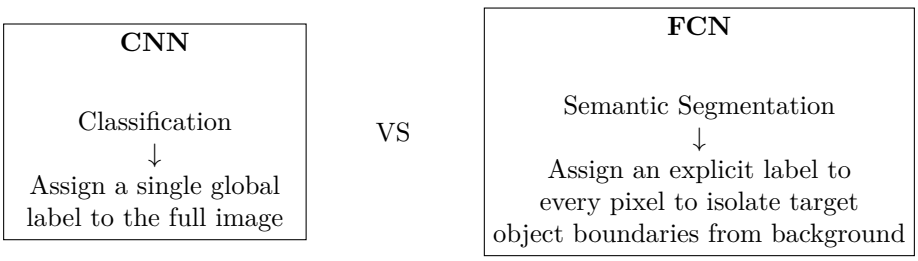
Solution:

1. Between the feature-extracting **convolutional** layers and the dense classification blocks **(fully connected)**, we implement a **flattening** layer that converts the multi-dimensional feature matrices into a 1D vector.
2. This unified vector is then branched into separate streams, routing into independent fully connected heads tailored to each objective (one for class prediction, another for spatial bounding box regression, etc.).

The independent objective metrics computed at each head are summed together to form a unified loss function. This allows the shared convolutional layers to optimize features that are highly effective for both tasks, while the specialized fully connected heads handle the distinct predictions.

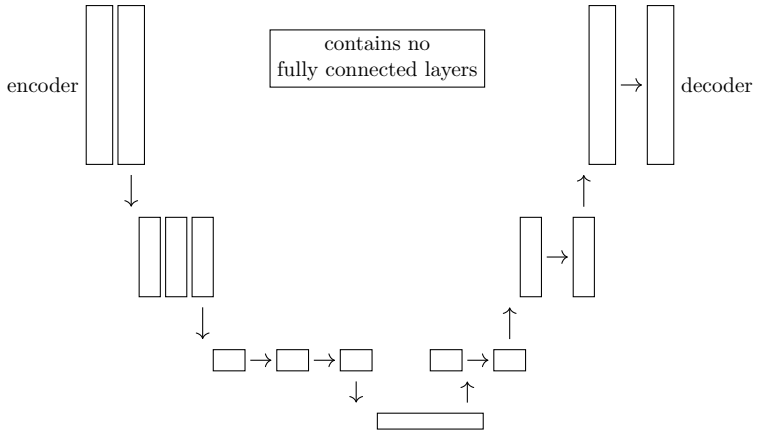
11 cnn vs fcn and u-net

There are two fundamentally different approaches for handling image processing tasks:



11.1 u-net

U-Net architectures are engineered specifically for precise semantic segmentation tasks. They ingest an input image and output a dense pixel map matching the exact spatial dimensions of the input, where each pixel denotes an object class.

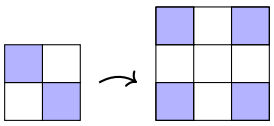


1. **Encoder:** A traditional convolutional pipeline paired with pooling layers to contract spatial dimensions while extracting high-level semantic features.
2. **Decoder:** An up-sampling pipeline that mirrors the encoder, utilizing two primary spatial expansion strategies:
    - Bilinear or Bicubic Interpolation
    - Transposed Convolutions (Up-Convolutions)

Crucially, the decoder implements **skip connections** to concatenate raw high-resolution feature maps from the contracting path directly into the expanding path. This mechanism restores precise localized spatial details that are routinely lost during pooling (e.g., the network can pinpoint exactly where an object sits spatially on the canvas, rather than just registering its global presence).

11.1.1 interpolation

Simple, deterministic up-sampling method that contains no trainable parameters.



- **Nearest Neighbor** $\rightarrow$ $\begin{array}{|c|c|}\hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \Rightarrow \begin{array}{|c|c|c|c|}\hline 1 & 1 & 2 & 2 \\ \hline 1 & 1 & 2 & 2 \\ \hline 3 & 3 & 4 & 4 \\ \hline 3 & 3 & 4 & 4 \\ \hline \end{array}$
- **Bilinear** $\rightarrow$ Computes a weighted average using the 4 closest neighboring pixels.
- **Bicubic** $\rightarrow$ Evaluates a broader 16-pixel neighborhood, applying a complex cubic interpolation.

11.1.2 transposed convolution / up convolution

Reversed pooling (unpooling) + standard convolution

- Spatial dimensions are expanded by interleaving zeros into the feature matrix, followed by padding adjustments tailored to the target kernel size.
- A learnable filter (e.g., 2×2) slides across this expanded grid, replacing sparse zeros with optimized, trainable parameter values.

11.2 u-net training mechanics

- For every single pixel coordinate on the canvas, the target label is structured as a dense one-hot vector $[0, 0, 1, 0]$ mapping its true object class. Following optimization, pixels tracking identical objects yield highly correlated features.

Weight Initialization Strategies:

- **Glorot / Xavier Initialization:** (Optimized for Sigmoid and Tanh activations)

Stabilizes gradient variance across layers

Uniform Distribution

$$\text{limit} = \sqrt{\frac{2 \cdot c}{\text{fan\_in} + \text{fan\_out}}}$$

Introduces bounded normal distribution constraints

Normal Distribution

$$\sigma = \sqrt{\frac{2 \cdot c}{\text{fan\_in} + \text{fan\_out}}}$$

- fan_in = number of incoming input neurons
- fan_out = number of outgoing output neurons
- c = user-defined scaling constant
- limit = bounding interval for weight sampling $[-\text{limit}, \text{limit}]$
- σ = standard deviation for normal sampling $[0, \sigma^2]$

- **He Initialization:** (Engineered specifically for ReLU activations)

Applies a similar variance-matching logic:

Uniform Distribution

$$\text{limit} = \sqrt{\frac{2 \cdot c}{\text{fan\_in}}} \quad [-\text{limit}, \text{limit}]$$

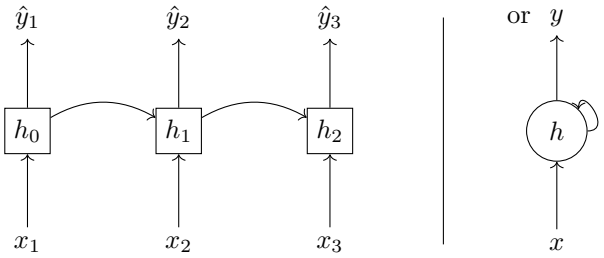
Normal Distribution

$$\sigma = \sqrt{\frac{2 \cdot c}{\text{fan\_in}}} \quad [0, \sigma^2]$$

12 rnn (recurrent neural network)

Recurrent Neural Networks are specialized architectures designed for processing sequential data streams like natural text, where predictions are contextually determined by combining the current input step with information from prior steps.

12.1 structural layout:



The core innovation is that while traditional feed-forward architectures implement independent weights per layer, recurrent layers h **share identical parameter matrices** across all time steps, enabling the system to evaluate inputs in relation to past context.

12.2 Architecture and Logic of Classical RNNs

Recurrent Neural Networks (RNNs) are fundamentally structured to ingest sequential data arrays where the precise temporal ordering of elements contains vital context (e.g., natural language text or financial time series). Unlike traditional *Feed-Forward* models, RNNs maintain an internal feedback loop that allows historical information to persist across steps.

This recurrent processing loop centers around the concept of a dynamic **hidden state** h_t, which functions as a transient short-term memory buffer. At each specific time step t, the network processes the current token x_t and blends it with the memory of the immediate prior step h_{t-1} using the following structural transition equation:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b) \tag{40}$$

The engineering power of this layout lies in **parameter sharing**: the weight matrices W_h (governing historical state routing) and W_x (governing current feature extraction), along with the bias vector b, remain **perfectly identical** across all processing steps in the sequence. Rather than instantiating new layers for every sequential token, the network reuses this exact algebric block cyclically, allowing it to process and generalize contextual dependencies regardless of sequence length.

12.3 bptt: (back propagation through time)

BPTT follows the foundational principles of traditional backpropagation, with a few temporal modifications.

Conceptual Pipeline:

$$\begin{array}{rcccc} & x_1 & x_2 & x_3 & \\ \text{input:} & \text{I} & \text{love} & \text{cats} & \\ \text{prediction:} & \text{I} & \text{love} & \text{cats} & \\ & y_1 & y_2 & y_3 & \dots \end{array}$$

- **Isolate and compute individual loss metrics at each output step:**

$$\begin{array}{ccccc} \hat{y}_1 & & \hat{y}_2 & & \hat{y}_3 \\ \uparrow & & \uparrow & & \uparrow \\ h_1 & \longrightarrow & h_2 & \longrightarrow & h_3 \\ \uparrow & & \uparrow & & \uparrow \\ x_1 & & x_2 & & x_3 \end{array} \quad \begin{array}{l} \\ \\ \text{loss}(\hat{y}_1, y_1) \\ \\ \text{loss}(\hat{y}_2, y_2) \\ \\ \text{loss}(\hat{y}_3, y_3) \end{array}$$

- **Accumulate local updates for recurrent weight h by stepping backward through time:**

Stepping backward chronologically (where current updates are bound to subsequent output states):

$$\begin{aligned} \Delta w_{h_3} &= \text{loss}(\hat{y}_3, y_3) \\ \Delta w_{h_2} &= \text{loss}(\hat{y}_2, y_2) + \text{loss}(\hat{y}_3, y_3) \\ \Delta w_{h_1} &= \text{loss}(\hat{y}_1, y_1) + \text{loss}(\hat{y}_2, y_2) + \text{loss}(\hat{y}_3, y_3) \end{aligned}$$

This accumulation explicitly evaluates the partial derivatives of each localized loss component with respect to the shared weight matrix.

13 vanishing and exploding gradients in sequence modeling

Because recurrent cells continuously reapply the exact same weight matrix to combine historical state data with new incoming features at every step:

$$h_t = W_h \cdot \overbrace{h_{t-1}}^{\text{history}} + W_x \cdot \overbrace{x_t}^{\text{input}} \tag{41}$$

If we unroll this recurrent step backward through time, we observe that the historical state is repeatedly multiplied by the same recurrent weight matrix:

$$h_t = W_h \cdot (W_h \cdot (W_h \cdot h_{t-3})) = W_h^3 \cdot h_{t-3} \tag{42}$$

This exponential term W_h^3 implies that if the dominant eigenvalue λ of matrix W_h evaluates to:

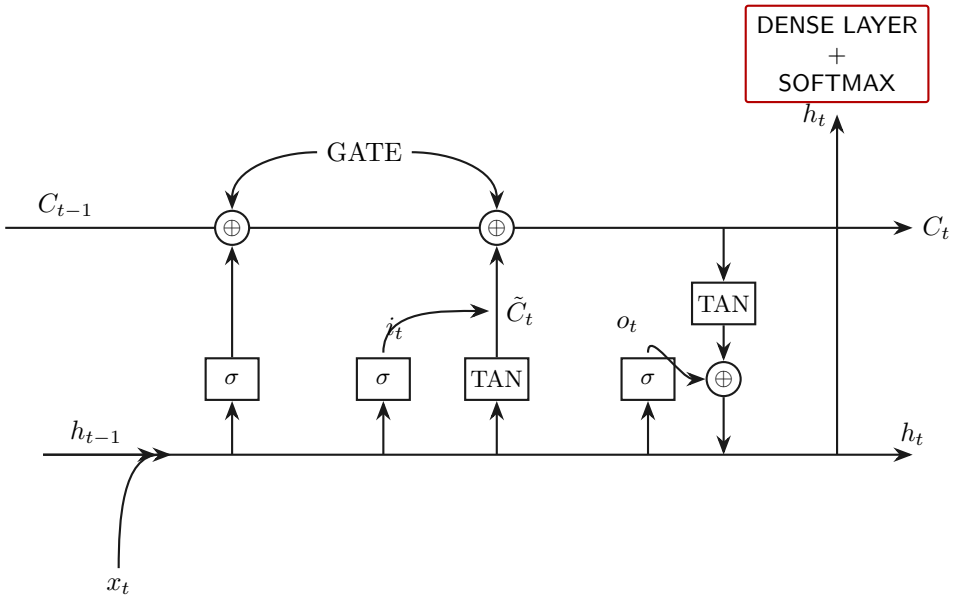
- $\lambda < 1 \rightarrow$ The error signal decays exponentially toward zero (Vanishing Gradient).
- $\lambda > 1 \rightarrow$ The error signal scales up exponentially (Exploding Gradient).

To overcome these fundamental mathematical constraints, sequence modeling architectures evolved from standard RNNs to sophisticated **LSTM** configurations.

14 lstm: (long short-term memory)

LSTM networks introduce an advanced gated architecture that enables the model to dynamically regulate information flow—deciding exactly what fraction of historical memory to discard, what new features to preserve, and how to structure the current output prediction.

INDIVIDUAL LSTM CELL ARCHITECTURE:



The hidden track h_t splits: One branch propagates into the subsequent cell step, while the other processes upward to compute localized error gradients and loss metrics.

14.1 forget gate: (regulating memory deactivation)

$$f_t = \sigma \cdot (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- f_t = activation output of the forget gate
- σ = Sigmoid function (yielding a strict $[0, 1]$ interval) that maps exactly what percentage of historical data to drop.
- W_f = parameter weight matrix governing the forget gate layer.
- $[h_{t-1}, x_t]$ = concatenated matrix pairing the prior hidden state with the current token input.
- b_f = bias parameter vector assigned to the forget gate.

We merge prior short-term memory explicitly with the fresh input token to guarantee highly informed context filtering (determining exactly what to clear based on new details).

14.2 input gate: (isolating incoming information value)

Employs an identical structural algebraic layout as the forget gate, varying exclusively across weight and bias configurations.

$$\boxed{i_t = \sigma \cdot (W_i \cdot [h_{t-1}, x_t] + b_i)}$$

- i_t = activation profile of the input gate
- σ = Sigmoid layer
- W_i = weight tensor tracking incoming feature relevance.
- $[h_{t-1}, x_t]$ = concatenated feature array.
- b_i = input gate bias vector.

Core Intuition: The vector i_t measures the network's localized filtering capacity, tracking exactly how much of the new input to assimilate based on current contextual knowledge.

What target values do we scale using this input filter? We multiply this critical gate metric directly by the newly formulated candidate information state $\tilde{C}_t$:

$$\boxed{\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)}$$

- The key operational difference here is the use of a Hyperbolic Tangent activation (bounding features to a $[-1, 1]$ interval). This allows the network to execute negative adjustments. Because long-term memory (C_t) builds up via linear accumulation, enabling $\tilde{C}_t$ to pass negative values allows the system to actively decrement or modify memory values rather than merely stacking features upward.

The consolidated long-term memory state updates as follows:

$$\boxed{C_t = \underbrace{f_t}_{} \cdot \overbrace{C_{t-1}}^{\text{prior long-term memory}} + \underbrace{i_t \cdot \tilde{C}_t}_{\text{new contextual features}}}$$

14.3 output gate:

Evaluates what elements of the cell state to manifest as short-term memory based on the current input features, scaling the result by the updated long-term memory:

- Information exposure filter:

$$\boxed{o_t = \sigma \cdot (W_o \cdot [h_{t-1}, x_t] + b_o)}$$

- Memory scaling:

$$\boxed{\tilde{h}_t = \tanh(C_t)} \rightarrow \text{where } C_t \text{ represents the current long-term cell memory}$$

- Consolidated short-term hidden output:

$$\boxed{h_t = o_t \cdot \tilde{h}_t}$$

There are two primary paradigms for structuring text generation tasks:

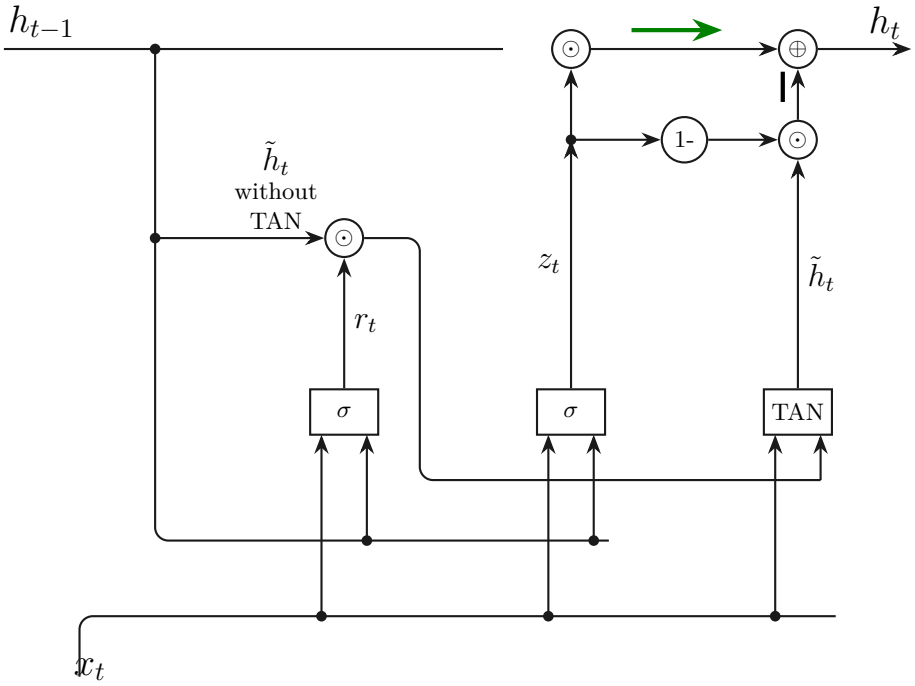
- **Character-Level Prediction:**
  - Evaluates a compact dictionary (typically 26 letters + around 100 symbols).
  - Lacks initial semantic awareness (must learn spelling rules from scratch).
  - Highly effective for automated text correction or highly granular generation.
- **Word-Level Prediction:**
  - Operates over massive dictionaries (frequently scaling past 50,000 distinct words).
  - Computationally expensive due to wide final projection layers.
  - Standard implementation for chatbots and conversational agents.

15 gru

Gated Recurrent Units (**GRU**) were developed to optimize and streamline the architectural complexity of traditional **LSTMs**. While LSTMs successfully mitigate vanishing gradients by introducing long-term cell states, they remain computationally expensive to train.

A **GRU** cell condenses the control structure down to just 2 gates:

- Reset Gate
- Update Gate



15.1 reset gate:

Determines exactly how much historical context remains relevant for computing the upcoming prediction step, managing **SHORT-TERM MEMORY**:

$$\boxed{r_t = \sigma \cdot (W_r \cdot x_t + U_r \cdot h_{t-1})}$$

- h_{t-1} = prior hidden state profile
- x_t = input feature vector at step t
- W_r, U_r = weight parameter matrices assigned to the reset gate
- σ = Sigmoid activation layer mapping features to a $(0, 1)$ interval

We then formulate the intermediate candidate state:

$$\boxed{\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}))}$$

(where $\odot$ denotes the element-wise Hadamard product).

CRITICAL NOTE: Although the mathematical layouts look highly similar across different gates, the decisive factor is the specialization of the weight matrices, which adapt during optimization to execute distinct tasks.

15.2 update gate:

Regulates exactly what percentage of historical memory h_{t-1} to retain and what fraction of the fresh candidate state $\tilde{h}_t$ to integrate, managing **LONG-TERM MEMORY**:

$$\boxed{z_t = \sigma \cdot (W_z \cdot x_t + U_z \cdot h_{t-1})}$$

- x_t = incoming input token features
- h_{t-1} = historical hidden state vector
- W_z, U_z = trainable parameter tensors assigned to the update gate
- σ = Sigmoid function

↓
Acts as a continuous interpolation balance weight between the reset gate output and the previous state.

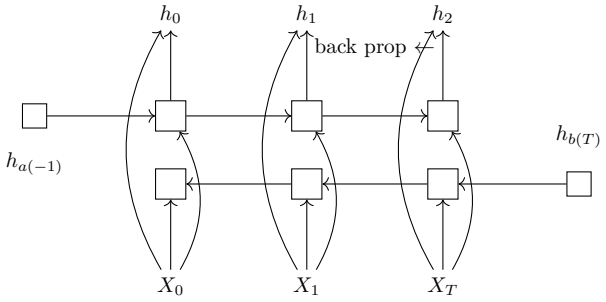
15.3 output computation:

Calculates the final hidden output state h_t leveraging the update vector z_t, seamlessly balancing immediate localized token details with historical context:

$$\boxed{h_t = \underbrace{(1 - z_t) \odot \tilde{h}_t}_{\text{The Present}} + \underbrace{z_t \odot h_{t-1}}_{\text{The Past}}}$$

16 bidirectional rnn

Bidirectional Recurrent Neural Networks are implemented when decoding a specific token requires evaluating future elements in the sequence. Standard recurrent structures are bound strictly to a chronological timeline, meaning they only process past context.



The terminal state h_0 combines information from the forward pass $h_{a(0)}$ and the backward pass $h_{b(0)}$. All individual layer vectors are computed independently across their respective timelines before being consolidated to form the final multi-directional context vector:

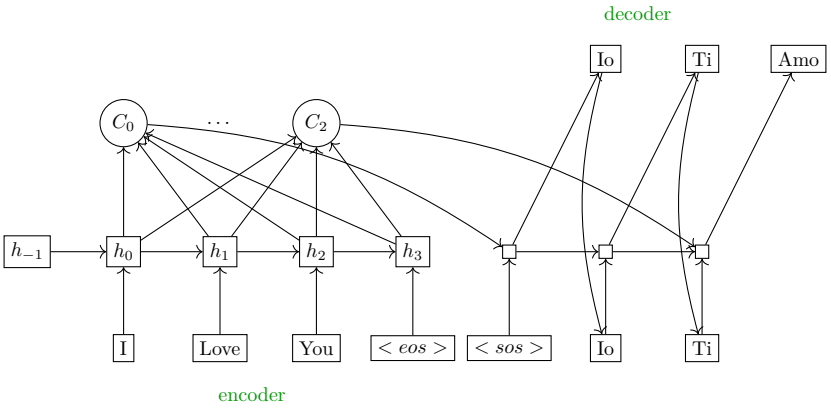
$$\boxed{h_{(t)} = [h_{a(t)}; h_{b(t)}]}$$

The backpropagation mechanics mirror classic sequential optimization, but execute two distinct backpropagation flows simultaneously (one tracking chronologically from end to start, and another tracking from start to end).

17 seq2seq with attention mechanisms

Sequence-to-Sequence models are highly effective for language translation tasks. They operate as asynchronously coupled modules: the network first ingests and completes the entire input sequence before initializing the output generation block. Within these setups, we implement an **Attention Mechanism** (C_t), which outputs a dynamic context vector designed to track the relationships across all hidden encoder states h_t.

17.1 architectural blueprint:



$$c_t = \frac{1}{N} \cdot \sum_i^N w_i(t) \cdot h_i$$

17.2 operational workflow steps:

1. Compute the sequential hidden state vectors h_t for each individual token, ingesting the preceding short-term state h_{t-1} at each step.

2. Derive the localized attention profile C_t, which manifests as an array of weights assigned to each h_t. This vector maps cross-token significance, tracking which features require focused attention based on the global context (requiring the module to evaluate the full collection of h_t vectors).

3. Once encoding completes, the features pass into the decoding block, where the core logic scales and weights each encoder state h_t against its corresponding attention score C_t.

4. Alongside these weighted encoder states, the decoding step ingests the text embedding of the token generated during the immediate prior cycle. This serves as an explicit placeholder or signpost, enabling the model to track its current generation index and apply correct syntactic rules.

5. To emit a valid output token, the system projects an internal state representation s_t (typically utilizing GRU-style cell states) into a final linear layer:

$$\text{Logits} = W_o \cdot s_t + b \tag{43}$$

   This generates a massive logit vector mapping probability scores across the entire vocabulary index.

17.3 Backpropagation Dynamics:

1. **Instantaneous Loss Calculation ($\mathcal{L}_t$):** The optimization sequence initializes at the termination of decoding step t. The system evaluates the discrepancy via a *Cross-Entropy Loss* function, comparing the Softmax probability vector with the true target token index.

2. **Backward Propagation Through the Output Projection Head:** The calculated error gradient flows backward through the Softmax normalization layer, isolating the error signals on the raw unnormalized

scores (*logits*). Next, it reverses across the final linear projection matrix W_o (the parameters mapping s_t), landing directly on the decoder's current hidden state vector:

$$\frac{\partial \mathcal{L}_t}{\partial s_t} = W_o^T \cdot \frac{\partial \mathcal{L}_t}{\partial z_t} \tag{44}$$

3. **Internal Gradient Bifurcation Within the Decoder Cell:** The hidden representation s_t consolidates features processed by the recurrent cell (such as GRU or LSTM). Upon entering this cell block, the error gradient splits along three distinct partial derivative paths:
   - Toward the prior hidden state s_{t-1}, stepping backward chronologically along the decoder timeline.
   - Toward the embedding vector of the previously generated token $\hat{y}_{t-1}$, updating vocabulary projection parameters.
   - Toward the context vector c_t, which serves as the physical link bridging the decoder back to the encoder.
4. **Gradient Routing Across the Attention Junction (c_t):** As the incoming error signal hits the context vector tracking node ($\frac{\partial \mathcal{L}_t}{\partial c_t}$), it encounters the linear attention weighting operator. Following the chain rule, the gradient signal divides into two simultaneous optimization trajectories:
   - **Alignment Optimization:** The gradient flows toward the stochastic weighting parameters $w_i(t)$, backing through the attention module's internal Softmax layer and score metrics. This directly optimizes the parameters that regulate the model's spatial alignment (refining its contextual focus).
   - **Injection Into the Encoder Path:** The gradient passes directly across the communication channel, scaled by its respective attention score, injecting error signals straight into the encoder hidden states:

$$\frac{\partial \mathcal{L}_t}{\partial h_i} = w_i(t) \cdot \frac{\partial \mathcal{L}_t}{\partial c_t} \tag{45}$$

5. **Encoder Gradient Accumulation:** Because any individual encoder state h_i may actively contribute to generating multiple distinct tokens across the decoder timeline, the total accumulated gradient impacting the i-th node is computed as the summation of all error signals back-channeled across all decoding time steps U:

$$\nabla_{h_i} \mathcal{L} = \sum_{t=1}^{U} \frac{\partial \mathcal{L}_t}{\partial h_i} \tag{46}$$

6. **Recurrent Backpropagation Along the Encoder Timeline:** Once the consolidated gradient totals are anchored across all h_i nodes, the system runs a standard Backpropagation Through Time loop backward along the encoder's axis ($i = T \rightarrow 1$). This terminal optimization phase drives updates to the internal recurrent parameter matrices ($W_{\text{enc}}, U_{\text{enc}}$) and the input layer embeddings associated with the original source sequence x_i.